

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
КАФЕДРА ИНТЕЛЛЕКТУАЛЬНЫХ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ

по лабораторной работе №6

Выполнил:

студент группы ПО-12

Снарский П.П.

Проверила:

Кулик А.Д.

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

Вариант 12

Ход работы

Задание 1.

Test_cart.py

```
"""
Тесты для мини-библиотеки покупок (shopping.py).
Запуск: pytest test_cart.py -v
"""

import pytest
from unittest.mock import patch, MagicMock
from shopping import Cart, log_purchase, apply_coupon

# -----
# Задание 1.1 – добавление товара
# -----

def test_add_item_increases_count(empty_cart):
    """После add_item в корзине должен быть ровно один элемент."""
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1

def test_add_item_stores_correct_data(empty_cart):
    """Убеждаемся, что имя и цена сохранены правильно."""
    empty_cart.add_item("Apple", 10.0)
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0

# -----
# Задание 1.2 – отрицательная цена
# -----

def test_add_item_negative_price_raises(empty_cart):
    """Отрицательная цена должна вызывать ValueError."""
    with pytest.raises(ValueError):
        empty_cart.add_item("Bad Item", -5.0)

# -----
# Задание 1.3 – общая стоимость
# -----

def test_total_empty_cart(empty_cart):
    """Сумма пустой корзины равна нулю."""
    assert empty_cart.total() == 0.0

def test_total_single_item(empty_cart):
    """Сумма корзины с одним товаром равна его цене."""
    empty_cart.add_item("Apple", 10.0)
    assert empty_cart.total() == 10.0
```

```

def test_total_multiple_items(empty_cart):
    """Сумма нескольких товаров считается верно."""
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.5)
    empty_cart.add_item("Cherry", 3.0)
    assert empty_cart.total() == pytest.approx(18.5)

# -----
# Задание 2 – apply_discount с @parametrize
# -----

@pytest.mark.parametrize(
    "discount, expected_price",
    [
        (0, 10.0), # 0 % – цена не меняется
        (50, 5.0), # 50 % – цена вдвое меньше
        (100, 0.0), # 100 % – цена обнуляется
    ],
)

def test_apply_discount_valid(empty_cart, discount, expected_price):
    """Корректные скидки изменяют цену ожидаемым образом."""
    empty_cart.add_item("Apple", 10.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.items[0]["price"] == pytest.approx(expected_price)

@pytest.mark.parametrize("bad_discount", [-1, -0.01, 101, 200])
def test_apply_discount_invalid_raises(empty_cart, bad_discount):
    """Скидка < 0 % или > 100 % должна вызывать ValueError."""
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(bad_discount)

# -----
# Задание 3: фикстура пустой корзины
# -----

@pytest.fixture
def empty_cart():
    """Возвращает свежий экземпляр Cart перед каждым тестом."""
    return Cart()

# -----
# Задание 4 – мок requests.post в log_purchase
# -----

def test_log_purchase_calls_post():
    """log_purchase должен вызывать requests.post с корректными данными."""
    item = {"name": "Apple", "price": 10.0}

    with patch("shopping.requests.post") as mock_post:
        log_purchase(item)

        mock_post.assert_called_once()

        args, kwargs = mock_post.call_args
        assert args[0] == "https://example.com/log"
        assert kwargs["json"] == item

```

```

def test_log_purchase_no_real_http():
    """Реальный HTTP-запрос НЕ должен уходить (мок перехватывает его)."""
    with patch("shopping.requests.post") as mock_post:
        mock_post.return_value = MagicMock(status_code=200)
        log_purchase({"name": "Test", "price": 1.0})

        assert mock_post.called

def test_apply_coupon_save10(empty_cart):
    """Купон SAVE10 даёт 10 % скидки."""
    empty_cart.add_item("Apple", 10.0)
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.items[0]["price"] == pytest.approx(9.0)

def test_apply_coupon_half(empty_cart):
    """Купон HALF даёт 50 % скидки."""
    empty_cart.add_item("Apple", 10.0)
    apply_coupon(empty_cart, "HALF")
    assert empty_cart.items[0]["price"] == pytest.approx(5.0)

def test_apply_coupon_invalid_raises(empty_cart):
    """Несуществующий купон должен вызывать ValueError."""
    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(empty_cart, "UNKNOWN")

def test_apply_coupon_with_patch_dict(empty_cart):
    """
    Мокаем модульный словарь COUPONS через patch.dict.
    Добавляем новый купон TESTCODE (20 %) и убеждаемся, что он работает,
    а оригинальный словарь не изменился после выхода из контекста.
    """
    import shopping

    empty_cart.add_item("Apple", 10.0)

    with patch.dict(shopping.COUPONS, {"TESTCODE": 20}, clear=False):
        apply_coupon(empty_cart, "TESTCODE") # 20 % от 10.0 → 8.0
        assert empty_cart.items[0]["price"] == pytest.approx(8.0)

        assert "SAVE10" in shopping.COUPONS
        assert "HALF" in shopping.COUPONS

    assert "TESTCODE" not in shopping.COUPONS

def test_apply_coupon_monkeypatch(empty_cart, monkeypatch):
    """
    Используем monkeypatch для подмены всей функции apply_coupon —
    убеждаемся, что тест изолирован от реального словаря купонов.
    """
    empty_cart.add_item("Apple", 10.0)
    custom_coupons = {"VIP": 30} # только наш купон

    def fake_apply_coupon(cart, coupon_code):
        if coupon_code in custom_coupons:
            cart.apply_discount(custom_coupons[coupon_code])
        else:

```

```

        raise ValueError("Invalid coupon")

import shopping

monkeypatch.setattr(shopping, "apply_coupon", fake_apply_coupon)

shopping.apply_coupon(empty_cart, "VIP")
assert empty_cart.items[0]["price"] == pytest.approx(7.0)

with pytest.raises(ValueError):
    shopping.apply_coupon(empty_cart, "SAVE10")

```

Shopping.py

```

"""Shopping module"""

import requests

class Cart:
    """Cart class"""

    def __init__(self):
        self.items = []

    def add_item(self, name: str, price: float):
        """Add an item to the cart"""
        if price < 0:
            raise ValueError(f"Price cannot be negative: {price}")
        self.items.append({"name": name, "price": price})

    def total(self) -> float:
        """Return the total price of the cart"""
        return sum(item["price"] for item in self.items)

    def apply_discount(self, percent: float):
        """Apply a discount for each item"""
        if percent < 0 or percent > 100:
            raise ValueError(f"Discount must be between 0 and 100, got {percent}")
        factor = 1 - percent / 100
        for item in self.items:
            item["price"] = round(item["price"] * factor, 10)

def log_purchase(item):
    """Log a purchase"""
    requests.post("https://example.com/log", json=item, timeout=5)

COUPONS = {"SAVE10": 10, "HALF": 50}

def apply_coupon(cart, coupon_code):
    """Apply a coupon"""
    coupons = COUPONS
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")

```

Задание 2

```

"""
Тесты для лабораторной работы №1.
Запуск: pytest test_lab1.py -v
"""

import pytest
from lab1 import generate_sequence, is_palindrome

class TestGenerateSequence:
    """Testing generating sequence."""

    def test_simple_ascending(self):
        """Стандартная возрастающая последовательность."""
        assert generate_sequence(1, 5, 1) == [1, 2, 3, 4]

    def test_simple_descending(self):
        """Стандартная убывающая последовательность."""
        assert generate_sequence(5, 1, -1) == [5, 4, 3, 2]

    def test_step_greater_than_one(self):
        """Шаг больше 1 — берём каждый второй элемент."""
        assert generate_sequence(0, 10, 2) == [0, 2, 4, 6, 8]

    def test_negative_step_greater_than_one(self):
        """Отрицательный шаг больше 1 по модулю."""
        assert generate_sequence(10, 0, -3) == [10, 7, 4, 1]

    def test_negative_range_positive_step(self):
        """Диапазон из отрицательных чисел, шаг положительный."""
        assert generate_sequence(-5, 0, 1) == [-5, -4, -3, -2, -1]

    def test_all_negative(self):
        """Оба конца отрицательные, убывание."""
        assert generate_sequence(-1, -5, -1) == [-1, -2, -3, -4]

    def test_start_equals_end(self):
        """start == end → пустой список независимо от шага."""
        assert not generate_sequence(3, 3, 1)
        assert not generate_sequence(3, 3, -1)

    def test_single_element(self):
        """Ровно один элемент: end = start + step."""
        assert generate_sequence(7, 8, 1) == [7]

    def test_step_larger_than_range(self):
        """Шаг больше длины диапазона → только start."""
        assert generate_sequence(0, 3, 10) == [0]

    def test_zero_start(self):
        """start = 0."""
        assert generate_sequence(0, 3, 1) == [0, 1, 2]

    def test_large_step_skips_all_but_start(self):
        """Шаг равен длине диапазона — ровно один элемент."""
        assert generate_sequence(1, 6, 5) == [1]

    def test_step_zero_raises(self):
        """Шаг 0 должен вызывать ValueError."""
        with pytest.raises(ValueError, match="нулю"):
            generate_sequence(1, 10, 0)

    def test_wrong_sign_positive_raises(self):
        """start < end, но step < 0 — недостижимый диапазон."""

```

```

        with pytest.raises(ValueError, match="Недостижимый"):
            generate_sequence(1, 10, -1)

    def test_wrong_sign_negative_raises(self):
        """start > end, но step > 0 — недостижимый диапазон."""
        with pytest.raises(ValueError, match="Недостижимый"):
            generate_sequence(10, 1, 1)

    @pytest.mark.parametrize(
        "start, end, step, expected",
        [
            (0, 0, 1, []), # пустой
            (0, 1, 1, [0]), # один элемент
            (0, 4, 1, [0, 1, 2, 3]), # четыре элемента
            (-3, 3, 2, [-3, -1, 1]), # шаг 2 через ноль
            (6, 0, -2, [6, 4, 2]), # убывание с шагом -2
        ],
    )
    def test_parametrized(self, start, end, step, expected):
        """Параметризованный тест"""
        assert generate_sequence(start, end, step) == expected


class TestIsPalindrome:
    """Testing palindrome"""

    def test_simple_true(self):
        """Простое слово-палиндром."""
        assert is_palindrome("madam") is True

    def test_simple_false(self):
        """Обычное слово — не палиндром."""
        assert is_palindrome("hello") is False

    def test_classic_phrase(self):
        """Классическая фраза с пробелами и знаками."""
        assert is_palindrome("A man a plan a canal Panama") is True

    def test_with_punctuation(self):
        """Знаки препинания игнорируются."""
        assert is_palindrome("No 'x' in Nixon") is True

    def test_mixed_case(self):
        """Регистр не важен."""
        assert is_palindrome("RaceCar") is True

    def test_numbers(self):
        """Цифровой палиндром."""
        assert is_palindrome("12321") is True

    def test_numbers_false(self):
        """Цифровая строка — не палиндром."""
        assert is_palindrome("12345") is False

    def test_mixed_alpha_and_digits(self):
        """Буквы и цифры вместе."""
        assert is_palindrome("alb2bla") is True

    def test_empty_string(self):
        """Пустая строка считается палиндромом."""
        assert is_palindrome("") is True

    def test_single_char(self):
        """Один символ — всегда палиндром."""

```

```

    assert is_palindrome("x") is True

def test_two_same_chars(self):
    """Два одинаковых символа."""
    assert is_palindrome("aa") is True

def test_two_different_chars(self):
    """Два разных символа."""
    assert is_palindrome("ab") is False

def test_only_spaces_and_punctuation(self):
    """Строка только из пробелов и знаков — пустая после очистки →
палиндром."""
    assert is_palindrome("   !!!   ") is True

def test_unicode_letters(self):
    """Unicode-буквы: isalnum() должен их учитывать."""
    assert is_palindrome("шалаш") is True

def test_long_non_palindrome(self):
    """Длинная строка — не палиндром."""
    assert is_palindrome("abcdefghij") is False

def test_none_raises_type_error(self):
    """None вместо строки → TypeError."""
    with pytest.raises(TypeError, match="str"):
        is_palindrome(None)

def test_int_raises_type_error(self):
    """Целое число вместо строки → TypeError."""
    with pytest.raises(TypeError, match="str"):
        is_palindrome(12321)

def test_list_raises_type_error(self):
    """Список вместо строки → TypeError."""
    with pytest.raises(TypeError):
        is_palindrome(["m", "a", "d", "a", "m"])

@pytest.mark.parametrize(
    "phrase, expected",
    [
        ("", True), # пустая строка
        ("a", True), # один символ
        ("aba", True), # нечётная длина
        ("abba", True), # чётная длина
        ("abcba", True), # нечётная длина 5
        ("abc", False), # не палиндром
        ("Madam", True), # регистр
        ("Was it a car or a cat I saw", True), # фраза с пробелами
        ("Never odd or even", True), # классический
        ("Not a palindrome", False), # явно не палиндром
    ],
)

def test_parametrized(self, phrase, expected):
    """Параметризованный тест"""
    assert is_palindrome(phrase) is expected

```

Задание 3

Кеер.py


```

"""
Реализация метода keep(str, pattern).

Спецификация:
    keep(None, None) → TypeError
    keep(None, *) → None
    keep("", *) → ""
    keep(*, None) → ""
    keep(*, "") → ""
    keep(" hello ", "hl") → " hll "
    keep(" hello ", "le") → " ell "
"""

def keep(s: str | None, pattern: str | None) -> str | None:
    """
    Оставляет в строке s только те символы, которые присутствуют в pattern.

    Args:
        s: Исходная строка (или None).
        pattern: Строка-фильтр (или None).

    Returns:
        - None, если s is None (и pattern не None).
        - "", если s == "" — независимо от pattern.
        - "", если pattern is None или pattern == "".
        - Отфильтрованную строку в остальных случаях.

    Raises:
        TypeError: если оба аргумента равны None.
    """
    # keep(None, None) → TypeError
    if s is None and pattern is None:
        raise TypeError("Оба аргумента не могут быть None одновременно.")

    # keep(None, *) → None
    if s is None:
        return None

    # keep("", *) → ""
    if s == "":
        return ""

    # keep(*, None) → "" | keep(*, "") → ""
    if pattern is None or pattern == "":
        return ""

    # Основная логика: оставляем только символы, входящие в pattern
    allowed = set(pattern)
    return "".join(c for c in s if c in allowed)

```

Test_keep.py

```

"""
Тесты для метода keep(str, pattern) — написаны ДО реализации (TDD).
Запуск: pytest test_keep.py -v
"""

import pytest
from keep import keep

def test_both_none_raises_type_error():
    """keep(None, None) → TypeError."""
    with pytest.raises(TypeError):

```

```

        keep(None, None)

@pytest.mark.parametrize("pattern", ["hl", "", "abc", " "])
def test_str_none_returns_none(pattern):
    """keep(None, *) → None (str=None, pattern – любой не-None)."""
    assert keep(None, pattern) is None

@pytest.mark.parametrize("pattern", ["hl", "", "abc", " ", None])
def test_empty_str_returns_empty(pattern):
    """keep(' ', *) → ' ' (пустая строка – любой pattern, включая None)."""
    # Исключение: keep(None, None) уже покрыто отдельным тестом,
    # здесь str всегда ' ', поэтому None-pattern допустим.
    assert keep("", pattern) == ""

@pytest.mark.parametrize("s", ["hello", "world", "test", " hello "])
def test_pattern_none_returns_empty(s):
    """keep(*, None) → ' ' (str не None, pattern=None)."""
    assert keep(s, None) == ""

@pytest.mark.parametrize("s", ["hello", "world", "test", " hello "])
def test_pattern_empty_returns_empty(s):
    """keep(*, ' ') → ' ' (pattern – пустая строка)."""
    assert keep(s, " ") == ""

def test_spec_example_hl():
    """keep(' hello ', 'hl') = ' hll ' – из спецификации."""
    assert keep(" hello ", "hl") == " hll "

def test_spec_example_le():
    """keep(' hello ', 'le') = ' ell ' – из спецификации."""
    assert keep(" hello ", "le") == " ell "

```

Вывод: в ходе выполнения лабораторной работы освоил приемы тестирования кода на примере использования пакета pytest